



CEVA-XM6

Gaussian 3x3 Lab

Dec, 2020

www.ceva-dsp.com



Copyright © 2019 CEVA, Inc.

Lab Agenda



- ❖ Lab Key Points
- ❖ Gaussian 3x3 Kernel Overview
- ❖ Lab Flow
- ❖ Gaussian 3x3 Vec-c implementation
- ❖ Gaussian 3x3 Vector Operators Code
 - ❖ Gaussian 3x3 Load Optimization
- ❖ Gaussian 3x3 Sliding Window Code
- ❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Agenda



❖ Lab Key Points

❖ Gaussian 3x3 Kernel Overview

❖ Lab Flow

❖ Gaussian 3x3 Vec-c implementation

❖ Gaussian 3x3 Vector Operators Code

❖ Gaussian 3x3 Load Optimization

❖ Gaussian 3x3 Sliding Window Code

❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Key Points

- ▶ In this lab you will implement a Gaussian 3x3 optimized kernel.
- ▶ Write and debug the code using visual studio.
- ▶ When writing a new kernel, as a first step you should usually write a simple c code that will be used as reference for comparing both the result and the performance of your kernel.
- ▶ In this lab the c reference code was already written for you.
- ▶ after writing the code and checking the validity of the results, the performance will be examined using IDE tools in eclipse environment

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Agenda



❖ Lab Key Points

❖ **Gaussian 3x3 Kernel Overview**

❖ Lab Flow

❖ Gaussian 3x3 Vec-c implementation

❖ Gaussian 3x3 Vector Operators Code

❖ Gaussian 3x3 Load Optimization

❖ Gaussian 3x3 Sliding Window Code

❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

**Gaussian 3x3
overview**

Lab Flow

Vec-c
implementation

Operators code

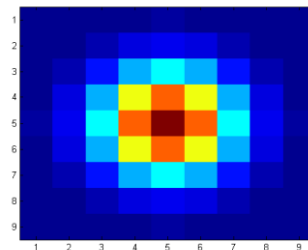
Sliding window

performance

Gaussian 3x3 Kernel Overview

- The Gaussian smoothing operator is a 2-D convolution operator that is used to 'blur' images and remove details and noise using a kernel that represents the shape of a Gaussian.

- Gaussian 3x3 coefficients:
$$\begin{bmatrix} 2 & 4 & 2 \\ 4 & 8 & 4 \\ 2 & 4 & 2 \end{bmatrix}$$



- $$Res(Xpos, Ypos) = \frac{1}{\sum_{i=0}^2 \sum_{j=0}^2 coeff(i,j)} \sum_{i=0}^2 \sum_{j=0}^2 Source(Xpos + i, Ypos + j) \cdot coeff(i,j)$$

Key Points

**Gaussian 3x3
overview**

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Gaussian 3x3 Kernel Overview

► C reference implementation:

```
for (i = 0; i < u32M; i++)
{
    uint acc = 0;
    acc += p_in_u8[-1 - s32SrcStride] * s8Kernel[0];
    acc += p_in_u8[0 - s32SrcStride] * s8Kernel[1];
    acc += p_in_u8[1 - s32SrcStride] * s8Kernel[2];
    acc += p_in_u8[-1] * s8Kernel[3];
    acc += p_in_u8[0] * s8Kernel[4];
    acc += p_in_u8[1] * s8Kernel[5];
    acc += p_in_u8[-1 + s32SrcStride] * s8Kernel[6];
    acc += p_in_u8[0 + s32SrcStride] * s8Kernel[7];
    acc += p_in_u8[1 + s32SrcStride] * s8Kernel[8];
    res = (acc >> ps) & 0xFFFF;
    sat = (res < 0) ? 0 : (res > 255) ? 255 : (uchar)res;
    p_out_u8[i] = sat;
    p_in_u8++;
}
```

- The reference code is based on fixed point arithmetic
- Use the reference to check the validity of your results
- At the final stage of examining the performance, compare the performance of the optimized kernel to the reference kernel.

Key Points

**Gaussian 3x3
overview**

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Agenda



❖ Lab Key Points

❖ Gaussian 3x3 Kernel Overview

❖ **Lab Flow**

❖ Gaussian 3x3 Vec-c implementation

❖ Gaussian 3x3 Vector Operators Code

❖ Gaussian 3x3 Load Optimization

❖ Gaussian 3x3 Sliding Window Code

❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Flow

► The lab folder (gaussian3x3) contains the following:

► cevaxm6

- Eclipse files - .cproject , .project

► Include

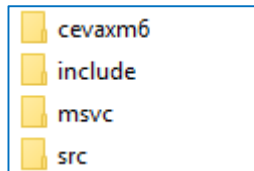
- All .h files needed for the project

► msvc

- Visual studio files - .sln, .vcproj, .vxproj.filters

► src:

- all .cpp files - **you should only edit gaussian3x3.cpp!!**
- gaussian3x3_xm6.cpp - a solution to the lab



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Flow



- ▶ The lab contains three coding exercises
 - ▶ Vector operator usage
 - ▶ Vector load optimization
 - ▶ Vector sliding window optimization
- ▶ The implementation will be done in gaussian3x3.cpp file
 - ▶ Contains the reference implementation and 3 empty functions:

```
void CCV_gaussian3x3_ref(uchar *p_u8Src, uchar *p_u8Dst, int s32SrcStep, int s32DstStep, uint u32N, uint u32M);  
  
void CCV_gaussian3x3_voperator(uchar * RESTRICT p_u8Src, uchar * RESTRICT p_u8Dst, int s32SrcStride, int s32DstStride, uint u32N, uint u32M);  
void CCV_gaussian3x3_vldov(uchar * RESTRICT p_u8Src, uchar * RESTRICT p_u8Dst, int s32SrcStep, int s32DstStep, uint u32N, uint u32M);  
void CCV_gaussian3x3_sliding_window(uchar * RESTRICT p_u8Src, uchar * RESTRICT p_u8Dst, int s32SrcStep, int s32DstStep, uint u32N, uint u32M);
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Flow



- ▶ The main() function was implemented in gaussian3x3_test.cpp . Its main stages:
 - ▶ Create a random image
 - ▶ Call CCV_gaussian3x3_ref and each one of the implementations (operator usage, load optimization, sliding window optimization).
 - ▶ Compare the results of each implementation to the reference.

```
init_gaussian3x3_test(p_u8Src);

memset(p_u8Dst, 0, sizeof(g_s8Dst_VP1));
memset(p_u8DstRef, 0, sizeof(g_s8Dst_VP2));

CCV_gaussian3x3_voperator(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);
CCV_gaussian3x3_ref(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8DstRef, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_voperator");

CCV_gaussian3x3_vldov(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_vldov");

CCV_gaussian3x3_sliding_window(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_sliding_window");
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Flow

- ▶ `main()` - executes the test reporting pass/fail by comparing the reference code and the user's code.
- ▶ In case of error the first mismatch in each line is displayed:
 - ▶ Fail

```
gaussian3x3 Test Number:0 : M = 128 ; N = 64
Error: function gaussian3x3_U8_U8 vec-c is not bit-exact to c [y,x] = [0,0] --> 0x0 != 0x69 (0x40000,0x40000) (0x60000,0x60000)
Error: function gaussian3x3_U8_U8 vec-c is not bit-exact to c [y,x] = [0,1] --> 0x0 != 0x8b (0x40000,0x40001) (0x60000,0x60001)
Error: function gaussian3x3_U8_U8 vec-c is not bit-exact to c [y,x] = [0,2] --> 0x0 != 0x9d (0x40000,0x40002) (0x60000,0x60002)
Error: function gaussian3x3_U8_U8 vec-c is not bit-exact to c [y,x] = [0,3] --> 0x0 != 0x8c (0x40000,0x40003) (0x60000,0x60003)
Error: function gaussian3x3_U8_U8 vec-c is not bit-exact to c [y,x] = [0,4] --> 0x0 != 0x7a (0x40000,0x40004) (0x60000,0x60004)
Error: function gaussian3x3_U8_U8 vec-c is not bit-exact to c [y,x] = [0,5] --> 0x0 != 0x7a (0x40000,0x40005) (0x60000,0x60005)
```

- ▶ Pass

```
gaussian3x3 Test Number:0 : M = 128 ; N = 64
function gaussian3x3_U8_U8 test: success
CEVA_CV lib: end test
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Flow

- Open the .sln file using visual studio and build the project.

```
1>----- Build started: Project: Gaussian3x3, Configuration: Debug Win32 -----
1> utilities.cpp
1> test.cpp
1> gaussian3x3_test.cpp
1> gaussian3x3.cpp
1> Generating Code...
1> Gaussian3x3.vcxproj -> \\giant_nas\Users\michals\my documents\CEVA-XM4_gaussian3x3_project\msvc\Debug\Gaussian3x3.exe
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```

- Make sure to run with Debug Build Configuration:

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Flow



- Note: in order to use vec-c , you should have the \$(CEVAXM6_SIMDIR) variable defined on your computer:

`$(CEVAXM6_SIMDIR)= C:\CEVA-ToolBox\Vxx\XM6\CEVA-XM6\fis\ceva-xm6`
*xx is the version number

To add a new macro:

Search for Advanced system settings on your control panel →
environment variables → new system variable

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Agenda



- ❖ Lab Key Points
- ❖ Gaussian 3x3 Kernel Overview
- ❖ Lab Flow
- ❖ **Gaussian 3x3 Vec-c implementation**
- ❖ Gaussian 3x3 Vector Operators Code
 - ❖ Gaussian 3x3 Load Optimization
- ❖ Gaussian 3x3 Sliding Window Code
- ❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Gaussian 3x3 Vec-c implementation

- **Vector types** – CEVA-XM compiler supports native vector types, which are mapped to the vector registers and can be used in vector operations:

8 integer vector

vX.i8															
i3				i2				i1				i0			
127		96	95		64	63		32	31		0				0
i7				i6				i5				i4			
255		224	223		192	191		160	159		128				128

16 short vector

vX.s16															
s7				s6				s5				s4			
127		112	111		96	95		80	79		64	63		48	47
s15				s14				s13				s12			
255		240	239		224	223		208	207		192	191		176	175

32 byte vector

vX.c32															
c15	c14	c13	c12	c11	c10	c9	c8	c7	c6	c5	c4	c3	c2	c1	c0
255		240	239		224	223		208	207		192	191		176	175
c31	c30	c29	c28	c27	c26	c25	c24	c23	c22	c21	c20	c19	c18	c17	c16
511		496	495		480	479		464	463		448	447		432	431

Native 256-bit vectors	
char32	uchar32
short16	ushort16
int8	uint8
128-bit vectors	
char16	uchar16
short8	ushort8
512-bit vectors	
short32	ushort32
int16	uint16

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Gaussian 3x3 Vec-c implementation

- ▶ **VEC-C** – vector intrinsic that have one to one mapping with the instruction set:
 - ▶ Enables to use CEVA-XM rich instruction set features from a C level environment.
 - ▶ Provides superb performance per invested coding time.
 - ▶ Documentation on all vec-c operations can be found in the installation package:
- ▶ `{root_folder}/CEVA-ToolBox/Vxx/XM6/help/CEVA-XM6/fis/navigator/index.html`

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

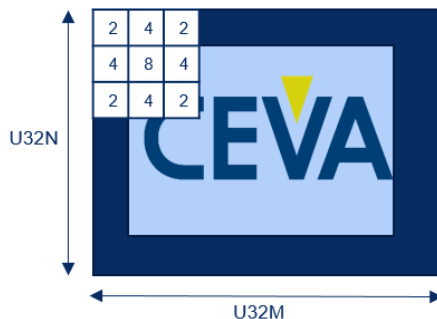
Sliding window

performance

Gaussian 3x3 Vec-c implementation

- Important notes:

- In this exercise ,each pixel size is 1byte (0-255)
- Vector size in VPU is 32bytes therefore each vector can store 32 pixels.
 - Vector type declaration example: `uchar32 Va;`
- The input image is already padded with zeros for the filter operation:



- When accumulating we might need to store bigger values therefore we use vector accumulator.
 - Vector accumulator example: `ushort32 Vacc;`

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Lab Agenda



- ❖ Lab Key Points
- ❖ Gaussian 3x3 Kernel Overview
- ❖ Lab Flow
- ❖ Gaussian 3x3 Vec-c implementation
- ❖ Gaussian 3x3 Vector Operators Code**
 - ❖ Gaussian 3x3 Load Optimization
- ❖ Gaussian 3x3 Sliding Window Code
- ❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

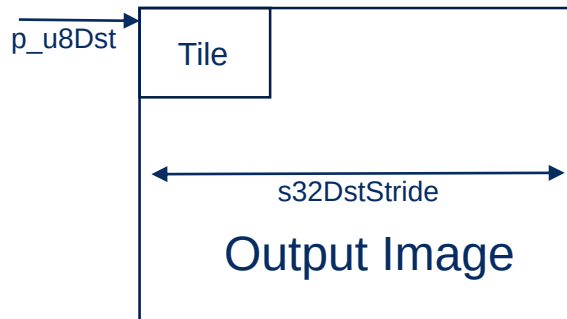
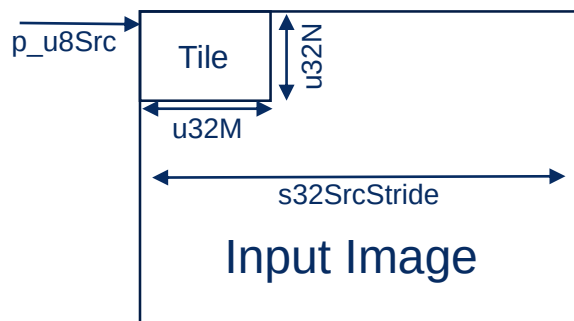
Sliding window

performance

Gaussian 3x3 Vector Operators Code

- At this section you will implement a Gaussian 3x3 kernel using vector operators.

```
/*  
*****  
// CVV_gaussian3x3_voperator - vec-c function of gaussian3x3  
//  
// inputs:  
// p_u8Src      - pointer to the source chunk  
// p_u8Dst      - pointer to the destination chunk  
// s32SrcStride - source stride represent the width + padding if applicable  
// s32DstStride - destination stride represent the width + padding if applicable  
// u32N         - number of rows in the tile  
// u32M         - number of columns in the tile, best performance if the number is divisible by 32, the code will treat leftovers.  
//*****  
void CVV_gaussian3x3_voperator(uchar * RESTRICT p_u8Src, uchar * RESTRICT p_u8Dst, int s32SrcStride, int s32DstStride, uint u32N, uint u32M)  
{
```



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Gaussian 3x3 Vector Operators Code

Vector operators – Simple coding of vector code, using predefined vector operators:

- ▶ Vector operators are used with the vector type as done for other native C language type [char, short or int].
- ▶ Ensures fast and simple code development
- ▶ Does not give access to all instruction set features

$$V_c = \underbrace{V_a + V_b}_{\text{vector operator}}$$

$$V_c = \underbrace{vadd(V_a, V_b)}_{\text{vec-c}}$$

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Gaussian 3x3 Vector Operators Code

load 9 input vectors:

- ▶ The code flow performs 9 vector loads

- ▶ Vector load example:

```
uchar32 va, vb, vc, vd;  
va = *(uchar32*)p_in_u8;  
p_in_u8++; //move to the next pixel  
vb = *(uchar32*)p_in_u8;  
p_in_u8++; // move to the next pixel  
vc = *(uchar32*)p_in_u8;  
p_in_u8 += (SrcStride-2); // move to the next row  
vd = *(uchar32*)p_in_u8;  
:
```



- ▶ Each load instruction loads 32 bytes into one vector
- ▶ Note the different pointer post-modification:
- ▶ Each calculation produces 32 output points

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

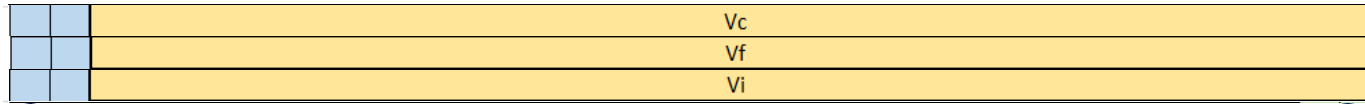
Operators
code

Sliding window

performance

Gaussian 3x3 Vector Operators Code

Calculate The filter:



$$\frac{1}{32} \cdot \begin{matrix} Va \cdot 2 + Vb \cdot 4 + Vc \cdot 2 \\ + \\ Vd \cdot 4 + Ve \cdot 8 + Vf \cdot 4 \\ + \\ Vg \cdot 2 + Vh \cdot 4 + Vi \cdot 2 \end{matrix}$$

```
Vacc = (ushort32)Va * 2 + (ushort32)Vb * 4;  
Vacc += (ushort32)Vc * 2 + (ushort32)Vd * 4;  
Vacc += (ushort32)Ve * 8 + (ushort32)Vf * 4;  
Vacc += (ushort32)Vg * 2 + (ushort32)Vh * 4;  
Vout = (uchar32)((Vacc + (ushort32)Vi * 2) >> 5);
```



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Gaussian 3x3 Vector Operators Code

- The algorithm flow you should implement is as follows:

```
for(each stripe in the image) {//each stripe is of size 32bytes  
    //calculate pointer to the beginning of the slice:  
    uchar* p_in_u8  = (uchar *)&p_u8Src[stripe * 32];  
    //calculate pointer to the destination slice  
    uchar* p_out_u8 = (uchar *)&p_u8Dst[stripe * 32];  
    for(each row in stripe) {  
        //load 9 input vectors: Va,Vb,Vc,Vd,Ve,Vf,Vg,Vh,Vi  
        //calculate the filter: Vout=result  
        //store the results:  
        vst(Vout, (uchar32*)p_out_u8, mask);  
    }  
}
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators
code

Sliding window

performance

Gaussian 3x3 Vector Operators Code

Important Notes:

- ▶ Kernel works on a tiles of size [M rows] x [N columns]. (example test tile size is 128X64)
- ▶ The Two nested loops that will be used:
 - ▶ Outer loop runs $\left\lfloor \frac{N+31}{32} \right\rfloor$ times $\rightarrow$ `uint iter_num = (N+31) >> 5`
 - ▶ Inner loop will work on a stripe of size [M rows] x [32 columns] $\rightarrow$ each iteration produces result for 32 pixels.
- ▶ The last stripe should be handled using predicate mask mechanism for the case where `last_stripe_width < 32`
- ▶ After multiplying & accumulating, the result should be normalized and truncated from ushort to uchar $\rightarrow$ `vout = uchar( $\frac{V_{acc}}{32}$ )`

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Gaussian 3x3 Vector Operators Code

Vector predicate mask calculation:

- ▶ The width of the last stripe might be smaller than 32
- ▶ We will use a special conditional store that takes only the valid pixels using a predicated mask vector.
- ▶ Vector predicate variable contains bit field flags that control which pixels will be stored
- ▶ The following code should be placed before the outer loop

```
uint u32OutLoop = (u32M + 31) >> 5; // calculates number of slices
unsigned int vprRightMask, vprMask; // Predicate definition
vprMask = 0xffffffff;                // default value, store all pixels

vprRightMask = 0xffffffff;           // default value, store all pixels

if (u32OutLoop != u32M >> 5)
    vprRightMask = (1 << (u32M & 31)) - 1;
```

- ▶ We use the following Vec-C intrinsic to implement a vector store with a predicate mask: `vst(vout, (uchar32*)p_out_u8, vprMask);`

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

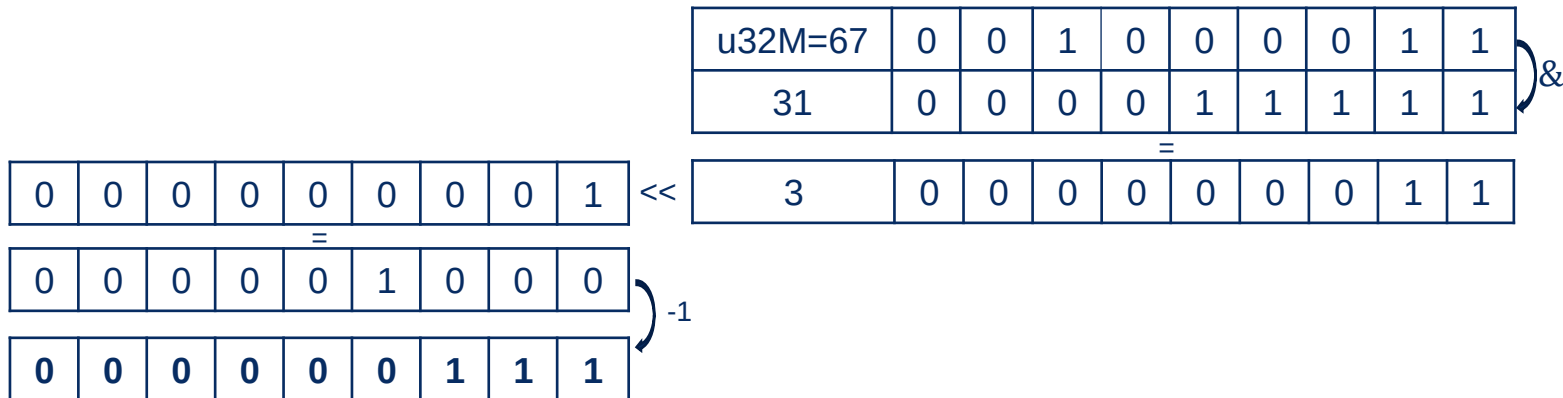
Sliding window

performance

Gaussian 3x3 Vector Operators Code

► Mask Handling Example- Image Size 67x128

```
uint u32OutLoop = (u32M + 31) >> 5;  $\rightarrow \left\lfloor \frac{(67 + 31)}{32} \right\rfloor = 3$   
if (u32OutLoop != u32M >> 5)  $\rightarrow 3 \neq \left\lfloor \frac{67}{32} \right\rfloor = 2$   
vprRightMask = (1 << (u32M & 31)) - 1;
```



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators
code

Sliding window

performance

Gaussian 3x3 Vector Operators Code



- ▶ After writing the code for `CCV_gaussian3x3_voperator` – build and run the project.
- ▶ Note: because you didn't implement `vldov` and `sliding_window` yet , you can put the calls to these function as a comment:

```
/*CCV_gaussian3x3_vldov(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);  
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_vldov");  
  
CCV_gaussian3x3_sliding_window(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);  
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_sliding_window");*/
```

- ▶ If your results are different from the reference code (not bit exact) - debug and find the problem before moving on to the next part of the lab.

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Lab Agenda



- ❖ Lab Key Points
- ❖ Gaussian 3x3 Kernel Overview
- ❖ Lab Flow
- ❖ Gaussian 3x3 Vec-c implementation
- ❖ Gaussian 3x3 Vector Operators Code
 - ❖ **Gaussian 3x3 Load Optimization**
- ❖ Gaussian 3x3 Sliding Window Code
- ❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Gaussian 3x3 Load Optimization

- ▶ When writing a kernel you should keep in mind that the DSP has the following limitations:
 - ▶ Two VPU units (VPU0 & VPU1)
 - ▶ Two LSU units (LSU0 & LSU1)
 - ▶ Only one store operation can be executed in one cycle.
- ▶ To help optimizing your kernel, use the following table, to predict its performance, and discover bottlenecks.

Cycles	LSU0	LSU1	VPU0	VPU1
1				
2				
3				
4				
5				

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Gaussian 3x3 Load Optimization

► So far- each iteration include:

- 9 vectorized load instruction to load pixel data
- 5 **vmac3** instructions (which can be executed on two VPU's!)

*Vmac3 = multiply and accumulate.

for example: $V_{acc} = \underbrace{Vb \cdot 4}_{\text{multiply}} + \underbrace{Vc \cdot 2}_{\text{multiply}} + \underbrace{+ V_{acc}}_{\text{accumulate}}$

Cycles	LSU0	LSU1	VPU0	VPU1
1	load	load	vmac3	vmac3
2	load	load	vmac3	vmac3
3	load	load	vmac3	
4	load	load		
5	load			

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators
code

Sliding window

performance

Gaussian 3x3 Load Optimization

► Optimization suggestion:

- Reduce the number of loads to get better balance with the vmac3 operations by using **vldov** operation

► **vldov** instruction

- Can load up to 4 vectors from addresses differing by a given offset **in only 1 cycle!**
- We will use a single **vldov** to load 3 vectors of 3 horizontal positions with offset of one pixel:

```
vldov((uchar32*)p_in_u8, Va, Vb, Vc, vdummy);
```



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Gaussian 3x3 Load Optimization

- Copy the code you implemented from `CCV_gaussian3x3_voperator` to `CCV_gaussian3x3_vldov`.
- Replace the regular loads by **vldov**.

```
va = *(uchar32*)p_in_u8;  
p_in_u8++;  
vb = *(uchar32*)p_in_u8;  
p_in_u8++;  
vc = *(uchar32*)p_in_u8;  
p_in_u8 += s32SrcStrideM2;  
vd = *(uchar32*)p_in_u8;  
p_in_u8++;  
ve = *(uchar32*)p_in_u8;  
p_in_u8++;  
vf = *(uchar32*)p_in_u8;  
p_in_u8 += s32SrcStrideM2;  
vg = *(uchar32*)p_in_u8;  
p_in_u8++;  
vh = *(uchar32*)p_in_u8;  
p_in_u8++;  
vi = *(uchar32*)p_in_u8;  
p_in_u8 -= s32SrcStrideP2;
```



```
vldov((uchar32*)p_in_u8, va, vb, vc, vdummy);  
p_in_u8 += s32SrcStep;  
vldov((uchar32*)p_in_u8, vd, ve, vf, vdummy);  
p_in_u8 += s32SrcStep;  
vldov((uchar32*)p_in_u8, vg, vh, vi, vdummy);  
p_in_u8 -= s32SrcStep;
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementationOperators
code

Sliding window

performance

Gaussian 3x3 Vector Operators Code

- ▶ Make sure that `CCV_gaussian3x3_vldov` is being called from `gaussian3x3_test.cpp` and run the project

```
CCV_gaussian3x3_ref(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8DstRef, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);

CCV_gaussian3x3_voperator(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_voperator");

CCV_gaussian3x3_vldov(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_vldov");

/*CCV_gaussian3x3_sliding_window(&p_u8Src[1 + 1 * BYTE_INSTRIDE], p_u8Dst, BYTE_INSTRIDE, BYTE_OUTSTRIDE, s32N, s32M);
cmp_test_result(p_u8Dst, p_u8DstRef, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, BYTE_OUTSTRIDE, HEIGHT, (uchar*)"CCV_gaussian3x3_sliding_window");*/
```

- ▶ If your results are different from the reference code (not bit exact) - debug and find the problem before moving on to the next part of the lab.

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

**Operators
code**

Sliding window

performance

Lab Agenda



- ❖ Lab Key Points
- ❖ Gaussian 3x3 Kernel Overview
- ❖ Lab Flow
- ❖ Gaussian 3x3 Vec-c implementation
- ❖ Gaussian 3x3 Vector Operators Code
 - ❖ Gaussian 3x3 Load Optimization
- ❖ **Gaussian 3x3 Sliding Window Code**
- ❖ Examine performance with CEVA tools(eclipse IDE)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

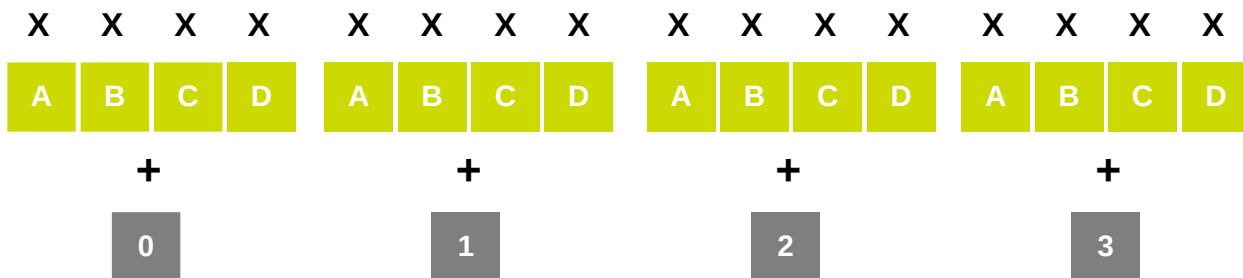
Operators code

**Sliding
window**

performance

Gaussian 3x3 Sliding Window Code

- ▶ The vector instruction performs the arithmetic operation multiple times in parallel (producing 8, 16 or 32 results).
- ▶ For each step the sliding window moves by an offset: (example with 4 coefficients)



Gaussian 3x3 Sliding Window Code

► Loading the filter coefficients:

```
ushort coeff[16] =  
{2,4,2,0,4,8,4,0,2,4,2,0,0,0,0,0};  
ushort16 v_coeff;  
v_coeff = *(ushort16*)coeff;
```

► Loading the pixel data:

```
uchar32 v00,v10,v20;  
uchar* p_in_u8 = (uchar*)&p_u8Src[i*16];  
v00 = *(uchar32*)p_in_u8;  
p_in_u8 += s32SrcStep;  
v10 = *(uchar32*)p_in_u8;  
p_in_u8 += s32SrcStep;  
v20 = *(uchar32*)p_in_u8;  
p_in_u8 += s32SrcStep;
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

**Sliding
window**

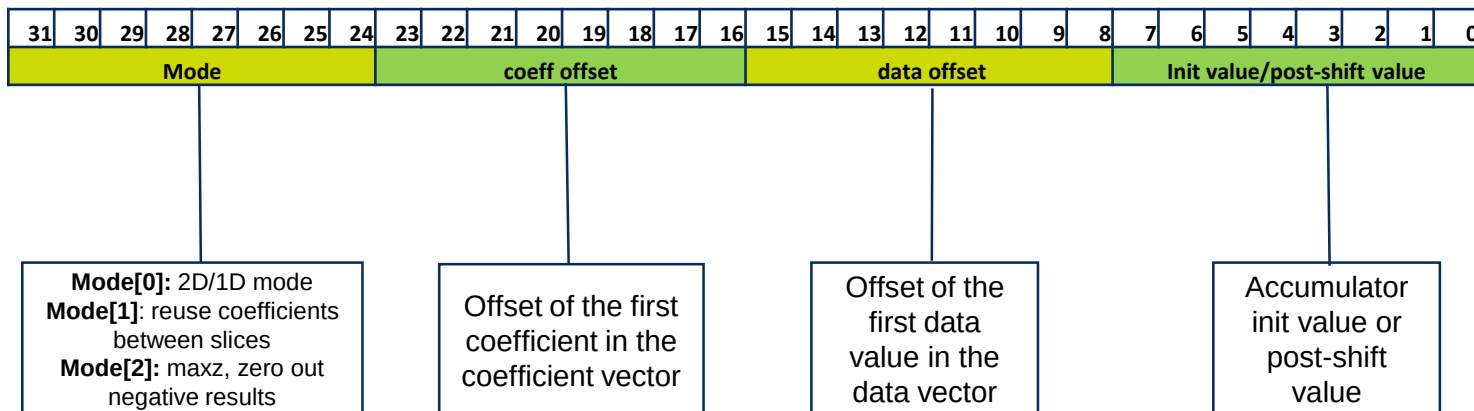
performance

Gaussian 3x3 Sliding Window Code

Sliding window intrinsic:

```
vacc0 = (uint16)vswgenmac(inputA, inputB, v_coeff1, v_coeff2, config0, config1, vacc0);
```

- config0,1 are control words - a 32 bit value, assigned to one of the scalar registers. This word defines the configuration of the instruction. **Config0** is:



Key Points

Gaussian 3x3 overview

Lab Flow

Vec-c implementation

Operators code

Sliding window

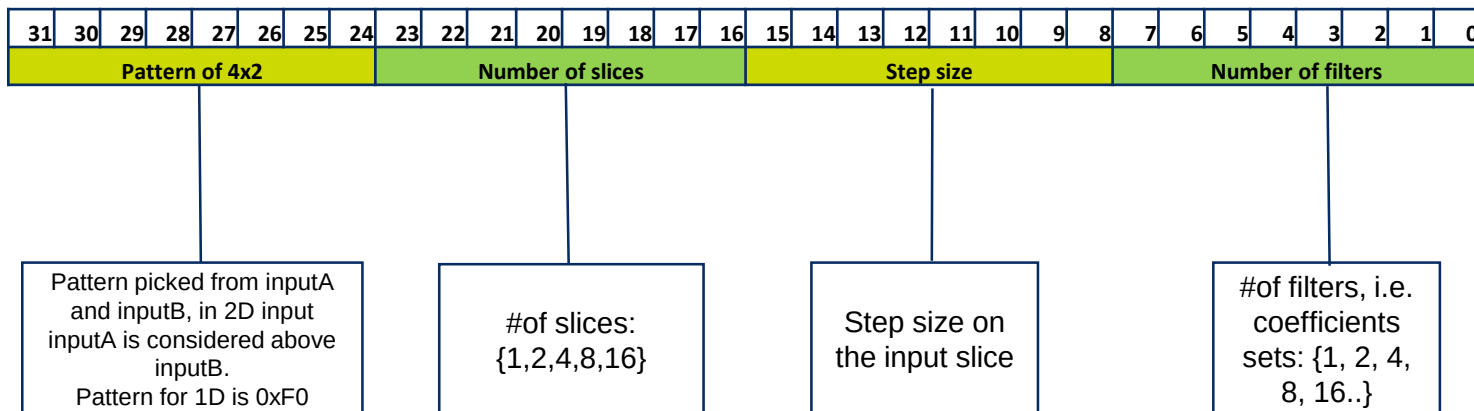
performance

Gaussian 3x3 Sliding Window Code

Sliding window intrinsic:

```
vacc0 = (uint16)vswgenmac(inputA, inputB, v_coeff1, v_coeff2, config0, config1, vacc0);
```

- config0,1 are control words - a 32 bit value, assigned to one of the scalar registers. This word defines the configuration of the instruction. **Config1** is:



Key Points

Gaussian 3x3 overview

Lab Flow

Vec-c implementation

Operators code

Sliding window

performance

Gaussian 3x3 Sliding Window Code

► First Sliding Window intrinsic will always have the **init switch** to clear the accumulator:

- `uint16 vswgenmac(init_sw, uchar32 inA, uchar32 inB, ushort16 inC, ushort16 inD, unsigned int inE, unsigned int inF);`
 - **Init_sw**: enable init to value (value in configuration word) by passing 'init'
 - **inA, inB** : input pixel vectors
 - **inC, inD** : 16 coefficients each (in Gaussian 3x3 only 9 will be non-zero)
 - **inE, inF** : configuration words

► Example

```
ushort coeff[16] = {2,4,2,0,4,8,4,0,2,4,2,0,0,0,0,0};
Ushort16 v_coeff = (ushort16) coeff;
uint config0 = SWGEN_CTRL0(0, 0, 0, SWGEN_BASE_MODE_1D); // 1D mode
uint config1 = SWGEN_CTRL1(FILTER_1, 0, SLICE_1, SWGEN_LINE); // 1D pattern
uint16 vacc0;
vacc0 = vswgenmac(init, v00, v00, v_coeff, v_coeff, config0, config1);
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

**Sliding
window**

performance

Gaussian 3x3 Sliding Window Code

- ▶ The next sliding window will use the **accumulate switch** to accumulate the current result with the previous one:
 - ▶ `uint16 vswgenmac(accumulate_sw, uchar32 inA, uchar32 inB, ushort16 inC, ushort16 inD, unsigned int inE, unsigned int inF, uint16 acc);`
- ▶ The **last sliding** window will use the **post-shift switch** to normalize the result:
 - ▶ `ushort16 vswgenmac(psl_sw, uchar32 inA, uchar32 inB, ushort16 inC, ushort16 inD, unsigned int inE, unsigned int inF, uint16 acc);`

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

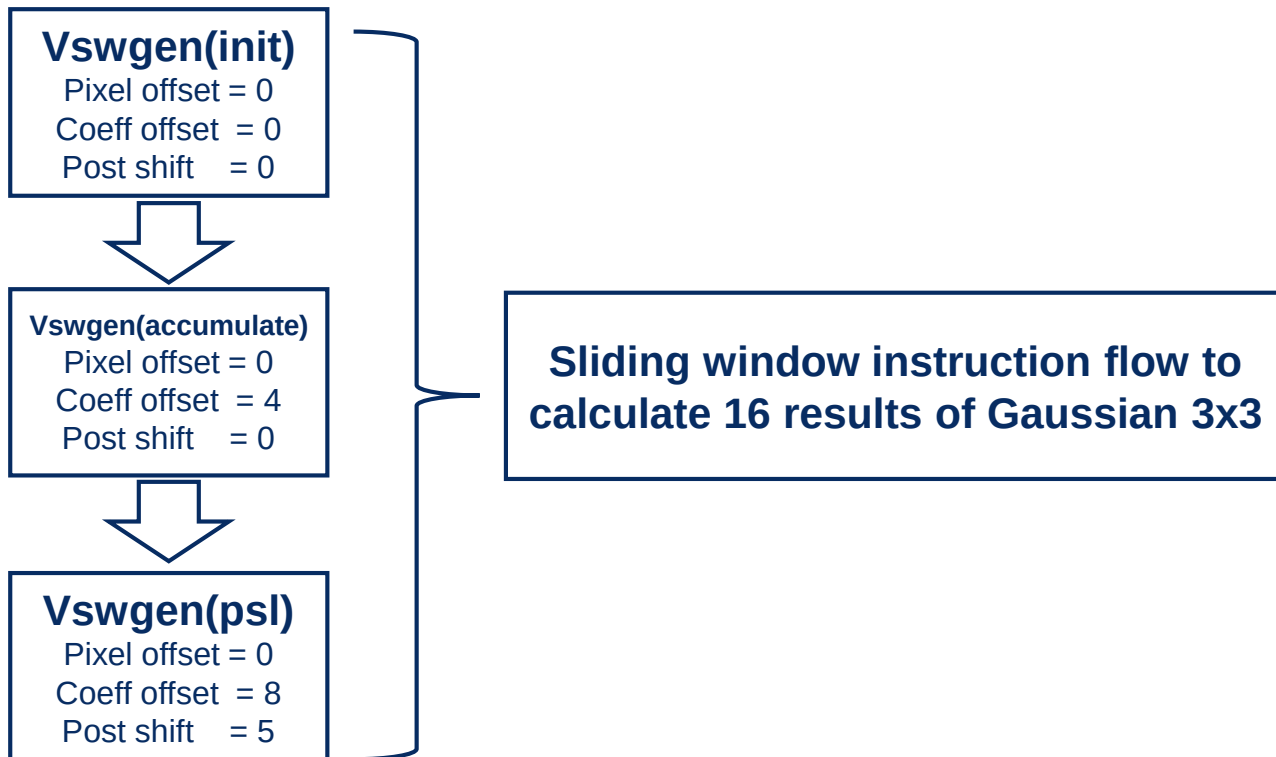
Operators code

**Sliding
window**

performance

Gaussian 3x3 Sliding Window Code

- Implementing 12 tap filter with three Sliding Window intrinsics:



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

**Sliding
window**

performance

Gaussian 3x3 Sliding Window Code

- ▶ Before the inner loop 3 lines must be pre-loaded to vectors: v0,v1,v2 (Scanning the image in the **vertical** direction with 1x32 **horizontal** vectors)
- ▶ Each iteration of the inner loop includes:
 - ▶ Processing: Each stage contains 3 sliding window instructions to process the data
 - ▶ Store: 1x16 result vector will be store.
 - ▶ Load:
 - ▶ Loading a new line to v2
 - ▶ **Please note: Before loading the data to v2, we must move the content of v1 to v0 and v2 to v1. The compiler will unroll the loop 2/4/8 times and eliminate these move command.**

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

**Sliding
window**

performance

Gaussian 3x3 Sliding Window Code

► Each iteration includes:

memory data																															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255
256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287
iteration 0																															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
iteration 1																															
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
iteration 1																															
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159

- Green marks the new data loaded
- Yellow marks the reused data
- The data load for iteration 0 done before entering the inner loop

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding
window

performance

Gaussian 3x3 Sliding Window Code

- Before you continue to the next part make sure that the gaussian3x3_test.cpp result for all of your implementations returns **success**

```
Start Testing gaussian3x3:  
  
gaussian3x3 Test Number:0 : M = 128 ; N = 64  
function CCV_gaussian3x3_voperator test: success  
function CCV_gaussian3x3_vldov test: success  
function CCV_gaussian3x3_sliding_window test: success
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

**Sliding
window**

performance

Lab Agenda



- ❖ Lab Key Points
- ❖ Gaussian 3x3 Kernel Overview
- ❖ Lab Flow
- ❖ Gaussian 3x3 Vec-c implementation
- ❖ Gaussian 3x3 Vector Operators Code
 - ❖ Gaussian 3x3 Load Optimization
- ❖ Gaussian 3x3 Sliding Window Code
- ❖ **Examine performance with CEVA tools(eclipse IDE)**

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Examine performance with CEVA tools



- ▶ After writing and validating your kernel in visual studio, you will now explore the performance of each implementation using eclipse simulation.
- ▶ The main steps for running simulation are:
 1. Import project to CEVA tools (eclipse IDE)
 2. Edit build configurations (o4 code optimization for Release)
 3. Edit debug configurations (enable profiling)

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

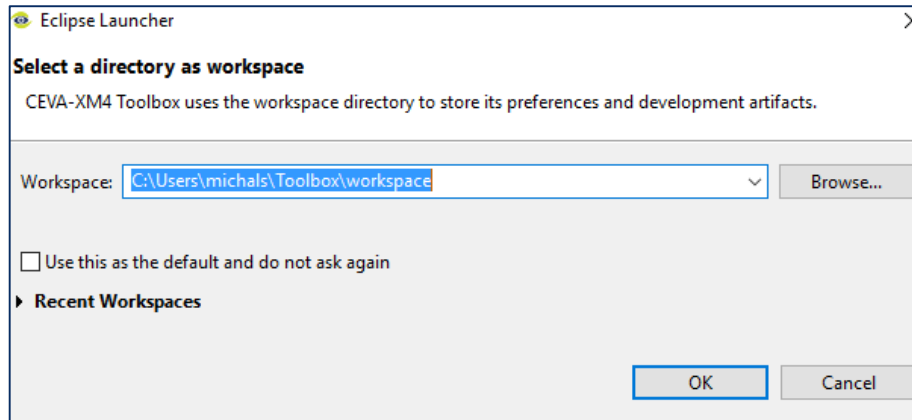
Operators code

Sliding window

performance

Examine performance with CEVA tools

- ▶ Import Project to CEVA Tools (Eclipse IDE)
 - ▶ Open the xm6 eclipse environment by click the following icon
 - ▶ Set the eclipse workspace. for example the default workspace:



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

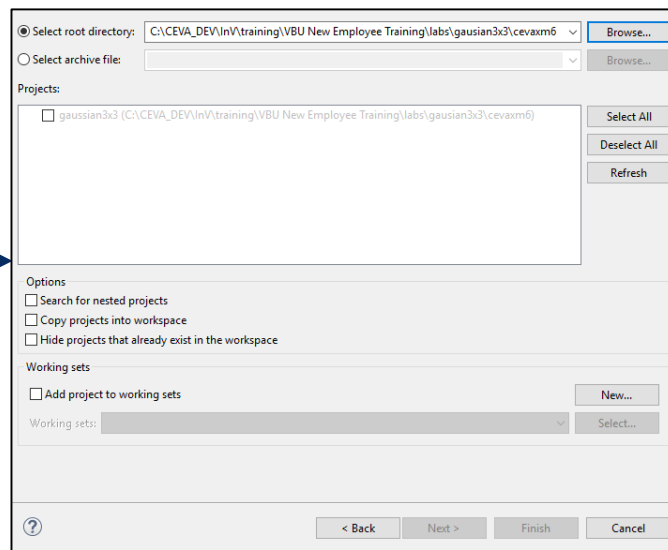
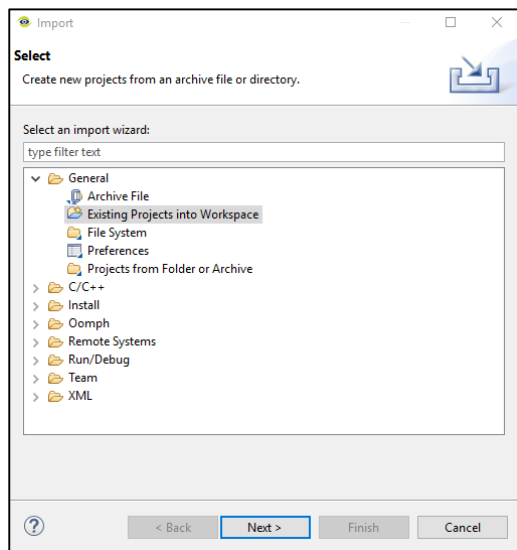
Sliding window

performance

Examine performance with CEVA tools

► Import the project:

- Right click on the project → Import → Existing Project into Workspace → Next → choose **gaussian3x3\cevaxm6** directory → Finish



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

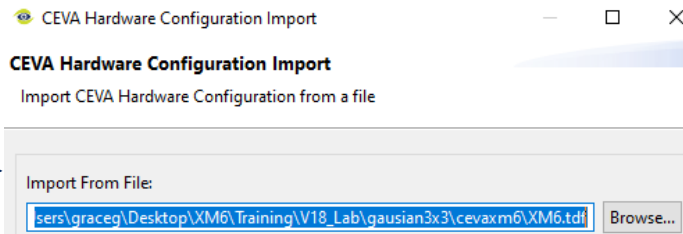
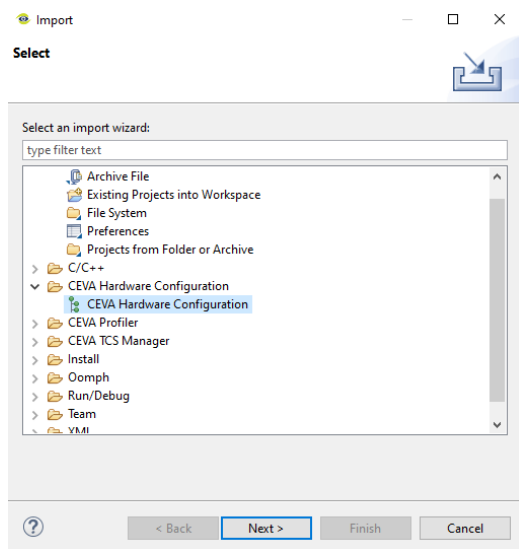
Sliding window

performance

Examine performance with CEVA tools

► Import the target file:

- Right click on the project → Import → CEVA Hardware Configuration → Next → choose **XM6.tdf** → Finish



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

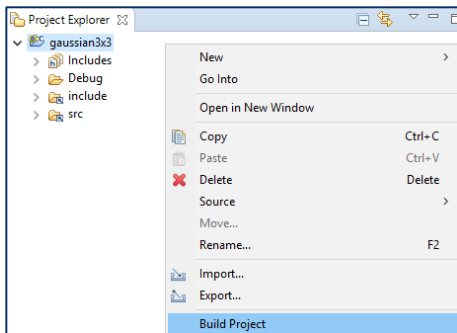
Sliding window

performance

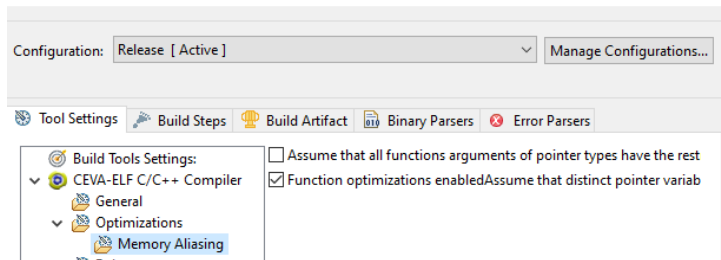
Examine performance with CEVA tools

► Building Running and profiling:

► Build the project: Right Click on project → Build Project



► Optimization under release mode



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Examine performance with CEVA tools



► Configure TCS manager

TCS Manager gaussian3x3_xm6.cpp

TCS Manager

▼ Drivers

+ Add - Remove Remove All

Name	Type
Driver	SIMULATION

Name	Value
------	-------

▼ Systems

+ Add - Remove Remove All

System	Driver	HW Config	State	Test	Activate
System	Driver	XM6		Test	☐
Cluster					
Core					

Name	Value
Simulation Built-In profiler config...	
Simulation Calling convention ch...	false
Simulation Exceptions	
Simulation Execution Mode	Performance Accurate Mode
Simulation MSS memory access	true
Simulation VCD trace XML	

► Connection

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

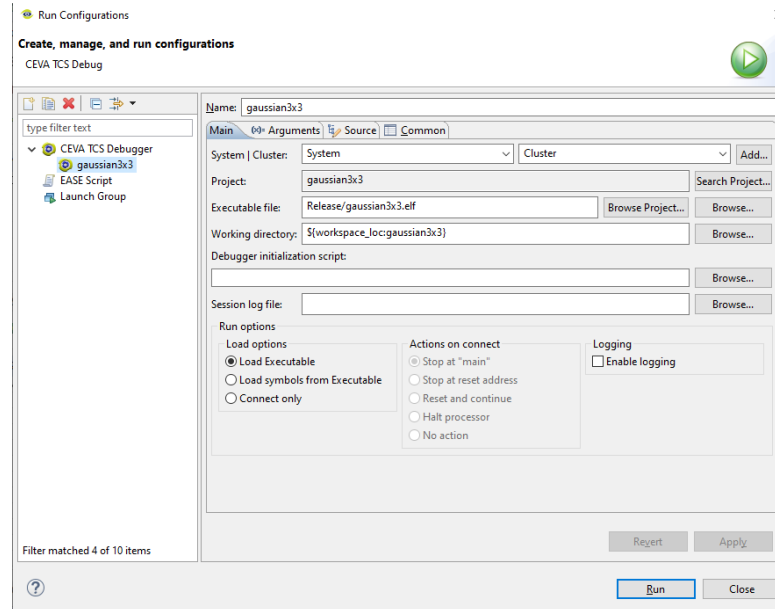
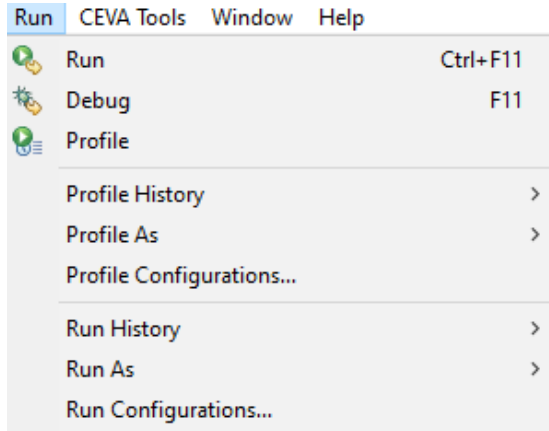
Operators code

Sliding window

performance

Examine performance with CEVA tools

- ▶ Create a CEVA TCS Debugger
- ▶ Use the search project button to add gaussian3x3
- ▶ Press apply and then Run



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Examine performance with CEVA tools



- Note That gaussian3x3_test.cpp result for all of your implementations is **success**

```
Start Testing gaussian3x3:  
  
gaussian3x3 Test Number:0 : M = 128 ; N = 64  
function CCV_gaussian3x3_voperator test: success  
function CCV_gaussian3x3_vldov test: success  
function CCV_gaussian3x3_sliding_window test: success  
CEVA_CV lib: end test
```

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

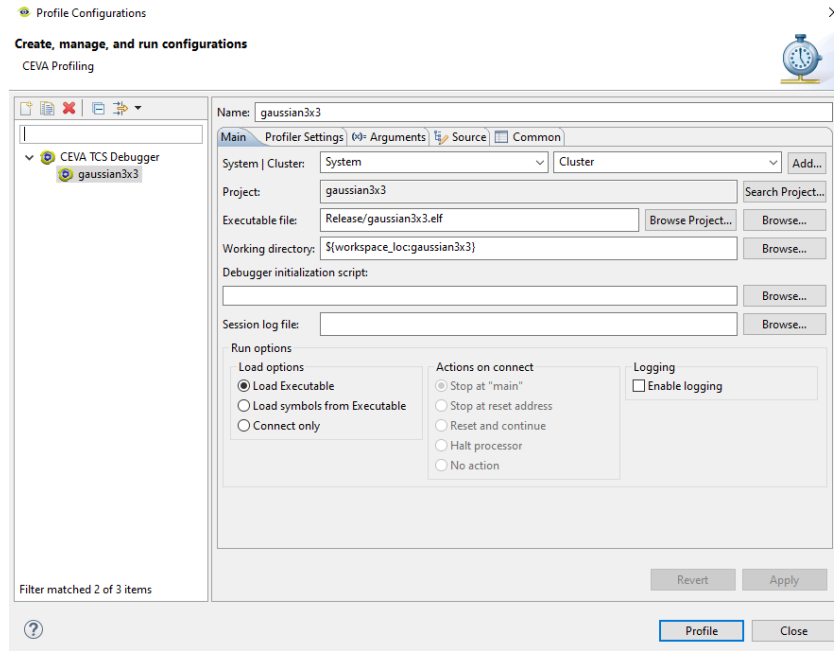
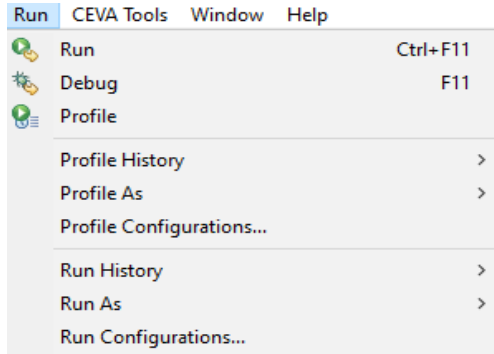
Sliding window

performance

Examine performance with CEVA tools



- ▶ Create a profiler configurations
- ▶ Press apply and then profiler



Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Examine performance with CEVA tools



- ▶ From the project explorer double click on **gaussian3x3_core0.prfdb** file in the **Debug** directory
- ▶ The following report will appear:

Name	Address	Size	Total Without Children	Total % Without Children	Total
main	0x40000000	118	35	.01	448313
CCV_gaussian3x3_test(int)	0x40000080	1354	330964	73.58	386145
CCV_gaussian3x3_ref(unsigned char*, unsigned ch	0x40000580	1994	28134	6.25	28134
CCV_gaussian3x3_voperator(unsigned char*, unsig	0x40000d20	2050	4222	.94	4222
CCV_gaussian3x3_vldov(unsigned char*, unsigned	0x40001520	848	4194	.93	4194
CCV_gaussian3x3_sliding_window(unsigned char*,	0x40001860	804	974	.22	974

Key Points

Gaussian 3x3
overview

Lab Flow

Vec-c
implementation

Operators code

Sliding window

performance

Thank You



www.ceva-dsp.com